CONCURRENCY
Mutual Exclusion and Synchronization

Multiple Processes

- OS design is concerned with managing multiple processes and threads
  - Multiprogramming
  - Multiprocessing
  - Distributed Processing
- Concurrency: Managing interaction of all these processes
- Concurrency encompasses a host of design issues
  - Communication among processes
  - Sharing of and competing for resources (such as memory, files, and I/O access),
  - Synchronization of the activities of multiple processes, and
  - Allocation of processor time to processes.

Cooperating Processes

- Concurrent processes /threads often need to share data (in shared memory or files) and resources
- If there is no controlled access to shared data, some processes will obtain an inconsistent view of this data
- The actions performed by concurrent processes may depend on the order in which their execution is interleaved
- With no synchronization, results are typically not deterministic nor reproducible.

Ex: UniProcessor - code in shared memory

```
void echo()
{
    chin = getchar();
    chout = chin;
    putchar(chout);
}
```

- Any program can call this procedure repeatedly to accept user input and display it on the user's screen.
- Sharing procedure in main memory for use by many programs

++ Save space

-- Create problems due to **shared global variable `chin`**

- P_1 invokes **`echo()`**, **`chin`**=x, interrupted after **`getchar()`**
- P_2 is activated, invokes **`echo()`**, runs to conclusion displaying y (**`chin = chout`** = y)
- When P_1 is resumed **`chin`**=y not x!

Enforce Single Access

- If we enforce a rule that only one process may enter the function at a time then:
  - P1 invokes **`echo()`**, **`chin`**=x, interrupted after **`getchar()`**
  - P2 is activated, invokes **`echo`**
  - Because P1 is still inside the **`echo`**, P2 is blocked from entering the procedure
  - P2 is suspended awaiting the availability of **`echo`** procedure
  - When P1 is resumed **`chin`**= x
  - P1 completes, unblock P2, P2 resumes and executes echo
- It is necessary to protect shared global variables/resources
- The only way to do that is to control the code accessing the variable/resource

Example: On a Multiprocessor

Process P1	**Process P2**
.	.
chin = getchar();	.
.	chin = getchar();
chout = chin;	chout = chin;
putchar(chout);	.
.	putchar(chout);
.	.

- on Multiptocessor system
  - same problem arise
  - character input to P1 is lost before being displayed and the character input to P2 is displayed by both P1 and P2
  - same solution works

Enforce Single Access

- If we enforce a rule that only one process may enter the function at a time then:
  - P1 & P2 run on separate processors
  - P1 enters echo() first,
  - P2 tries to enter but is blocked, waiting for echo()
  - P1 completes execution
  - P2 resumes and executes echo()

Race Condition

- A **race condition** occurs when
  - Multiple processes/threads read/write data items **concurrently**
  - They do so in a way where the final result **depends on the order** of execution of the processes.
- The output depends on who finishes the race last.
- Example
  - P1 and P2, share the global variable a.
  - At some point in its execution, P1 updates a to the value 1
  - At some point in its execution, P2 updates a to the value 2
  - Thus, the two tasks are in a race to write variable a.
  - the "loser" of the race (the process that updates last) determines the final value of a
- To guarantee correctness, processes must be **synchronized**

shared global variables – order matters

```
static int b=1, c=2;
void P3()                    void P4()
{                            {
    b = b + c;                       c = b + c;
}                            }
```

- P3 and P4 update different variables
- BUT: the final values of **b** and **c** depend on the order in which P3 and P4 execute these two assignments.
- If P3 executes its assignment statement first, then the final values are b = 3 and c = 5.
- If P4 executes its assignment statement first, then the final values are b = 4 and c = 3.

Operating Systems must…

- keep track of active processes, thru PCBs
- Allocate and deallocate resources for active processes, requesting same resources (processor time, memory, files, I/O devices)
- Protect the data and physical resources against interference by other processes
- The results of a process must be independent of the speed of execution relative to the speed of other concurrent processes.

It must provides tools for achieving **Mutual Exclusion**

Mutual Exclusion

- Suppose two or more processes require access to a single non-sharable resource, such as a printer
  - each process will be sending commands to the I/O device, receiving status information, sending and/or receiving data.
  - such a resource is called a **critical resource**, and the portion of the program that uses it is a **critical section**
    - Critical resource – a single non-sharable resource.
    - Critical section – portion of the program that accesses a critical resource
- only one program can be allowed in its critical section at a time (otherwise, lines from competing processes will be interleaved)
- Processes have to express requirement for mutual exclusion, not OS:
  - OS cannot understand and enforce this restriction because the detailed requirements are unknown to him
  - But OS has to provide support to processes e.g. Locking facility

Critical Section Problem

- Consider system of ***n*** processes {p_0, p_1, … p_{n-1}}
- Each process has **critical section** segment of code
  - Process may be changing common variables, updating table, writing file, etc
  - When one process in critical section, no other may be in its critical section
- ***Critical section problem*** is to design protocol for processes to cooperate
- Each process must ask permission to enter critical section in **entry section**, may follow critical section with **exit section**. The remaining code is the **remainder section**

General structure of process P_i

```
do {
    entry section
        critical section
    exit section
        remainder section
} while (true);
```

Mutual Exclusion Mechanism

```
/* PROCESS 1 */

void P1
{
  while (true) {
    /* preceding code /;
    entercritical (Ra);
    /* critical section */;
    exitcritical (Ra);
    /* following code */;
  }
}
```

```
/* PROCESS 2 */

void P2
{
  while (true) {
    /* preceding code */;
    entercritical (Ra);
    /* critical section */;
    exitcritical (Ra);
    /* following code */;
  }
}
```

. . .

```
/* PROCESS n */

void Pn
{
  while (true) {
    /* preceding code */;
    entercritical (Ra);
    /* critical section */;
    exitcritical (Ra);
    /* following code */;
  }
}
```

- There are *n processes to be executed concurrently.*
- Each process includes
  1. a critical section that operates on some resource Ra, and
  2. additional code preceding and following the critical section not needing Ra
- All processes want Ra→ only one process must be at a time in its critical section
- To enforce mutual exclusion, two functions are provided

  `entercritical`(Rx) and `exitcritical`(Rx)
- Any process trying to enter its critical section while another process is in its critical section, for the same resource, has to wait.

Sharing Global Variables – Data Coherence

```
static int a, b;
void P1()                    void P2()
{                            {
    a = a + 1;                       b = 2 * b;
    b = b + 1;                       a = 2 * a;
}                            }
```

- Assume a and b must be maintained in the relationship a=b
- If the state is initially consistent, each process should leave shared data consistent
- consider the following concurrent execution sequence

  a = a + 1;
  b = 2 * b;
  b = b + 1;
  a = 2 * a;

- Condition a=b no longer hold
- problem can be avoided by declaring the whole sequence a critical section
- Argument to `entercritical`(Rx) and `exitcritical`(Rx) could be a variable, a file or any shared object

Resource Competition

- Concurrent processes competing for same resource must leave the resources unaffected for use by other processes.
- The enforcement of mutual exclusion creates two control problems
- Deadlock
  - Each process owns a resource that the other is waiting for.
  - Two processes are waiting for communication from the other.
- Starvation
  - OS may grant access to resources to a number of processes while neglecting another
  - So, a process is denied access to a resource, even though there is no deadlock situation.

Some Definitions

- Race Conditions: two or more processes are reading and writing on shared data and the final result depends on who runs precisely when
- Mutual exclusion : making sure that if one process is accessing a shared resource, the other will be excluded from doing it
- Critical region: the part of the program where shared resources are accessed
- Atomic operation: A sequence of one or more statements that appears to be indivisible; that is, no other process can see an intermediate state or interrupt the operation.

Requirements for Mutual Exclusion

Any facility or capability that is to provide support for mutual exclusion should meet the following requirements:

1. Mutual exclusion must be enforced: Only one process at a time is allowed into its critical section, among all processes that have critical sections for same resource or shared object
2. A process that halts in its noncritical section must do so without interfering with other processes.
3. Must not be possible for a process requiring access to a critical section to be delayed indefinitely: no deadlock or starvation
4. When no process is in a critical section, any process requests entry to its critical section must be permitted without delay
5. No assumptions are made about relative process speeds or number of processors
6. A process remains in its critical section for a finite time only

Hardware Solutions: Disabling Interrupts

Preventing a process from being interrupted guarantees mutual exclusion

```
shared double balance;
```

Code for p_1

```
disableInterrupts();
  balance = balance + amount;
enableInterrupts();
```

Code for p_2

```
disableInterrupts();
  balance = balance - amount;
enableInterrupts();
```

Disadvantages

- Interrupts could be disabled arbitrarily long
- A user process can easily abuse this privilege and hence should not be available to a user process.
- We only want to prevent p_1 and p_2 from interfering with one another; this prevents any other process p_k to execute
- Does not work in a Multiprocessor system: disabling interrupts in one processor will not disable it in another process and hence mutual exclusion is not guaranteed

Mutual Exclusion: Hardware Support

- Many systems provide hardware support for implementing the critical section code.
- All solutions below based on idea of **locking:** Protecting critical regions via locks

```
do{
  acquire lock
        critical section
  release lock
        remainder section
} while (TRUE);
```

- Modern machines provide special atomic hardware instructions
  - **Atomic** = non-interruptible
  - Test memory word and set value `test_and_set()`
  - swap contents of two memory words `compare_and_swap()`

test_and_set Instruction

- **Instruction Definition**:

```
boolean test_and_set (boolean *target)
    {
        boolean rv = *target;
        *target = TRUE;
        return rv:
    }
```

1. Set the new value of passed parameter to “TRUE”.
2. Returns the original value of passed parameter
3. Executed atomically

- **Solution using `test_and_set()`**
  - Shared Boolean variable lock, initialized to FALSE

```
do {
        while (test_and_set(&lock))
        ; /* do nothing */
            /* critical section */
    lock = false;
            /* remainder section */
} while (true);
```

compare_and_swap Instruction

- Also called a “exchange instruction”
- A **compare** is made between a memory value and a test value
- If the values are the same a **swap** occurs
- Carried out atomically (not subject to interruption)

compare_and_swap Instruction

- **Definition:**

```
int compare_and_swap(int *value, int expected, int new_value) {
      int temp = *value;
      if (*value == expected)
         *value = new_value;
      return temp;
  }
```

1. Set the variable "value" the value of the passed parameter "new_value" but only if "value" =="expected". That is, the swap takes place only under this condition.
2. Returns the original value of passed parameter "value"
3. Executed atomically

- **Solution using `compare_and_swap`**
  - Shared integer "lock" initialized to 0;

```
do {
  while (compare_and_swap(&lock, 0, 1) != 0)   ; /* do nothing */
      /* critical section */
   lock = 0;
      /* remainder section */
  } while (true);
```

Special Machine Instruction: Pros & Cons

- **Advantages**
  - Applicable to any number of processes (uniprocessor or multiple processors) sharing main memory
  - Simple and easy to verify
  - It can be used to support multiple critical sections; each critical section can be defined by its own variable
- **Disadvantages**
  - Busy-waiting is employed
    - Thus while a process is waiting for access to a critical section it continues to consume processor time
  - Starvation is possible
    - When a process leaves a critical section and more than one process is waiting, the selection of a waiting process is arbitrary; some process could indefinitely be denied access
  - Deadlock is possible

Mutex Locks

- Hardware-based solutions are complicated and generally inaccessible to application programmers
  - OS designers build software tools to solve critical section problem
- Simplest is **mutex** lock
  - Protect a critical section to prevent race conditions
  - A process first `acquire()` a lock then `release()` it when it exits the critical section
- A mutex lock has a Boolean variable indicating if lock is available or not

```
acquire() {
  while (!available)
    ; /* busy wait */
  available = false;;
}
release() {
  available = true;
}
```

```
do {
    acquire lock
        critical section
    release lock
      remainder section
 } while (true);
```

acquire() and release()

- Calls to acquire() and release() must be atomic
  - Usually implemented via hardware atomic instructions
- **Disadvantage**: this solution requires **busy waiting**: While a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the call to **`acquire()`**.
  - This lock therefore called a **spinlock** because the process "spins" while waiting for the lock to become available
  - Busy waiting wastes CPU cycles that some other process might be able to use productively
- **Advantage**: no context switch is required when a process must wait on a lock
  - when locks are expected to be held for short times, spinlocks are useful.
  - They are often employed on multiprocessor systems where one thread can "spin" on one processor while another thread performs its critical section on another processor.

OS and Programming Language Mechanisms

- Another approach is to provide level of support to provide concurrency (mutual exclusion) within the OS or the programming language
  - **Semaphores**
  - **Monitors**
  - **Message Passing**

Semaphore

- **Semaphore**: An integer value used for signalling among processes
- Only three operations may be performed on a semaphore, all of which are atomic:
  - Initialize
    - A semaphore may be initialized to a nonnegative integer value
  - Decrement (`semWait`)
    - If the value becomes negative, then the process executing the semWait is blocked
    - Otherwise, the process continues execution
  - Increment (`semSignal`)
    - If the resulting value is less than or equal to zero, then a process blocked by a semWait operation, if any, is unblocked

Semaphore

- Synchronization tool that provides more sophisticated ways (than Mutex locks) for process to synchronize their activities.
- Semaphore ***S*** is an integer variable that can only be accessed via two atomic operations
  - **`wait()`** and **`signal()`**
- Definition of the **`wait()`** **operation**

```
wait(S) {
    while (S <= 0)
        ; // busy wait
    S--;
}
```

- Definition of the **`signal()`** **operation**

```
signal(S) {
    S++;
}
```

Semaphore

- To begin, the semaphore has a zero or positive value
  - If the value is positive, that value equals the number of processes that can issue a **`wait`** and immediately continue to execute (concurrent processes)
- If the value is zero, the next process to issue a **`wait`** is blocked, and the semaphore value goes negative
- Each subsequent **`wait`** drives the semaphore value further into minus territory
- The negative value equals the number of processes waiting to be unblocked
- Each **`signal`** unblocks one of the waiting processes when the semaphore value is negative

Semaphore Usage

- **Counting semaphore** – integer value can range over an unrestricted domain
- **Binary semaphore** – integer value can range only between 0 and 1
  - Same as a **mutex lock.** A key difference: the process that locks the mutex (sets the value to zero) must be the one to unlock it (sets the value to 1). In contrast, it is possible for one process to lock a binary semaphore and for another to unlock it
- Counting semaphores can be used to control access to a given resource consisting of a finite number of instances.
- Can solve various synchronization problems
- Consider two concurrent processes P_1 with a statement S_1 and P_2 with a statement S_2. S_1 is required to be completed before S_2
  - Create a semaphore "**synch**" initialized to 0

```
P1:                          P2:
    S1;                          wait(synch);
    signal(synch);               S2;
```

  - Because synch is initialized to 0, P_2 will execute S_2 only after P_1 has invoked **signal(synch)**, which is after statement S_1 has been executed

Semaphore Implementation

- Must guarantee that no two processes can execute the **`wait()`** and **`signal()`** on the same semaphore at the same time
- Thus, the implementation becomes the critical-section problem where the **`wait`** and **`signal`** code are placed in the critical section
  - Hardware based solutions e.g. **`compare_and_swap()`** is used
  - Could now have busy waiting in critical section implementation
    - But implementation code is short
    - Little busy waiting if critical section rarely occupied
- Note that applications may spend lots of time in critical sections and therefore this is not a good solution

Semaphore Implementation with no Busy waiting

- With each semaphore there is an associated waiting queue
- Each entry in a waiting queue has two data items:
  - value (of type integer)
  - pointer to next record in the list
- Two operations:
  - **block** – place the process invoking the operation on the appropriate waiting queue
  - **wakeup** – remove one of processes in the waiting queue and place it in the ready queue
-

```
typedef struct{
 int value;
 struct process *list;
 } semaphore;
```

Implementation with no Busy waiting (Cont.)

```
wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();
    }
}
signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);
    }
}
```

Definition of Semaphore Primitives

```
struct semaphore {
      int count;
      queueType queue;
};
void semWait(semaphore s)
{
      s.count--;
      if (s.count < 0) {
            /* place this process in s.queue */;
            /* block this process */;
      }
}
void semSignal(semaphore s)
{
      s.count++;
      if (s.count <= 0) {
            /* remove a process P from s.queue */;
            /* place process P on ready list */;
      }
}
```

Binary Semaphore

- A binary semaphore may only take the values 0 and 1

1. A binary semaphore may be initialized to 0 or 1
2. The semWaitB op checks the semaphore value:
   - If value=0, the process executing the semWaitB is blocked
   - If value=1, then the value is set to zero; the process continues execution
3. The semSignalB op checks to see if any processes are blocked on this semaphore
   - If so, then a process blocked by a semWaitB is unblocked
   - If no processes are blocked, then the value of the semaphore is set to 1

Binary Semaphore Primitives

```
struct binary semaphore {
      enum {zero, one} value;
      queueType queue;
};
void semWaitB(binary semaphore s)
{
      if (s.value == one)
            s.value = zero;
      else {
                /* place this process in s.queue */;
                /* block this process */;
      }
}
void semSignalB(semaphore s)
{
      if (s.queue is empty())
            s.value = one;
      else {
                /* remove a process P from s.queue */;
                /* place process P on ready list */;
      }
}
```

Mutual Exclusion Using Semaphores

```
/* program mutualexclusion */
const int n = /* number of processes  */;
semaphore s = 1;
void P(int i)
{
     while (true) {
          semWait(s);
          /* critical section   */;
          semSignal(s);
          /* remainder   */;
     }
}
void main()
{
     parbegin (P(1), P(2), . . ., P(n));
}
```

Processes Using Semaphore

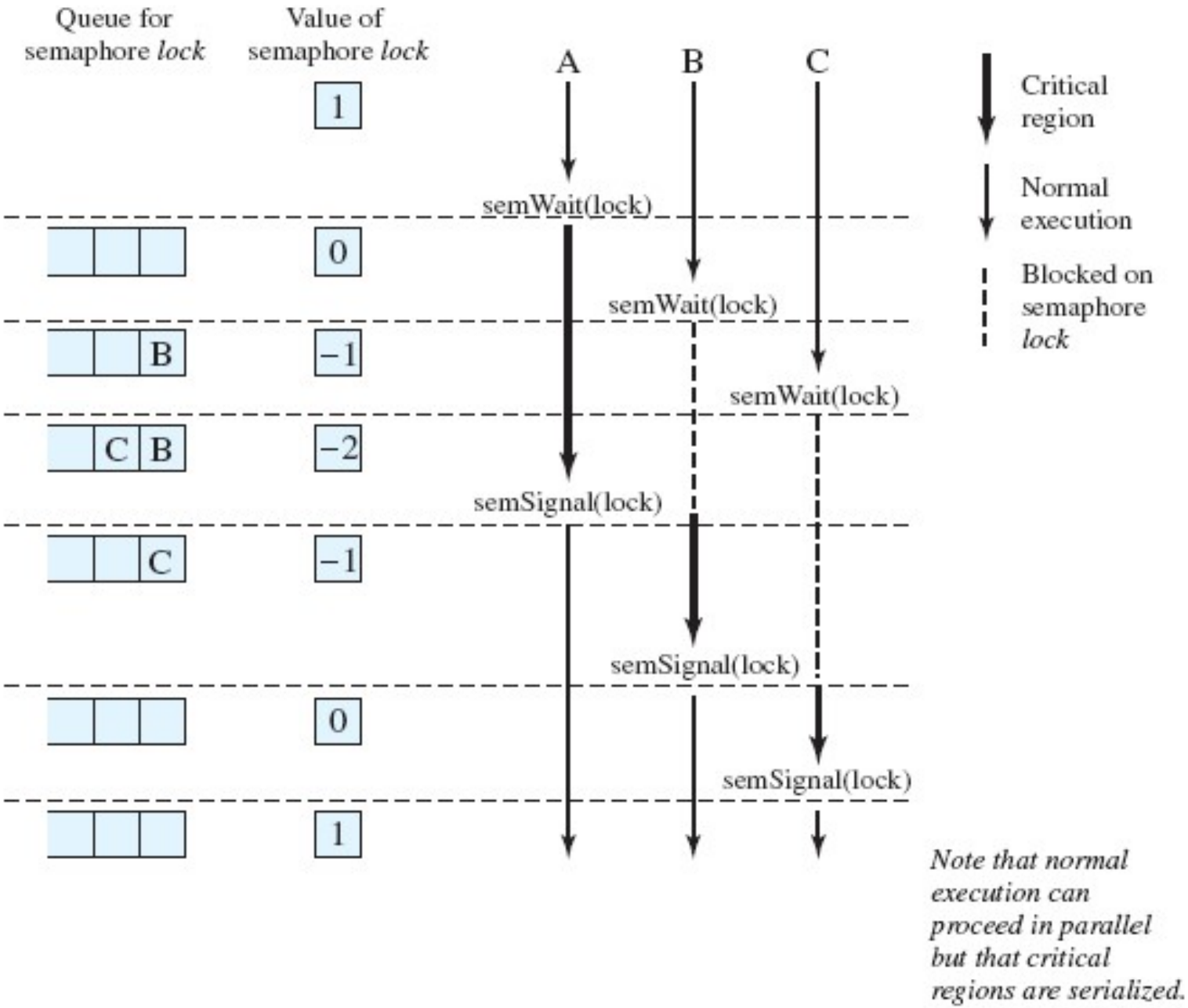


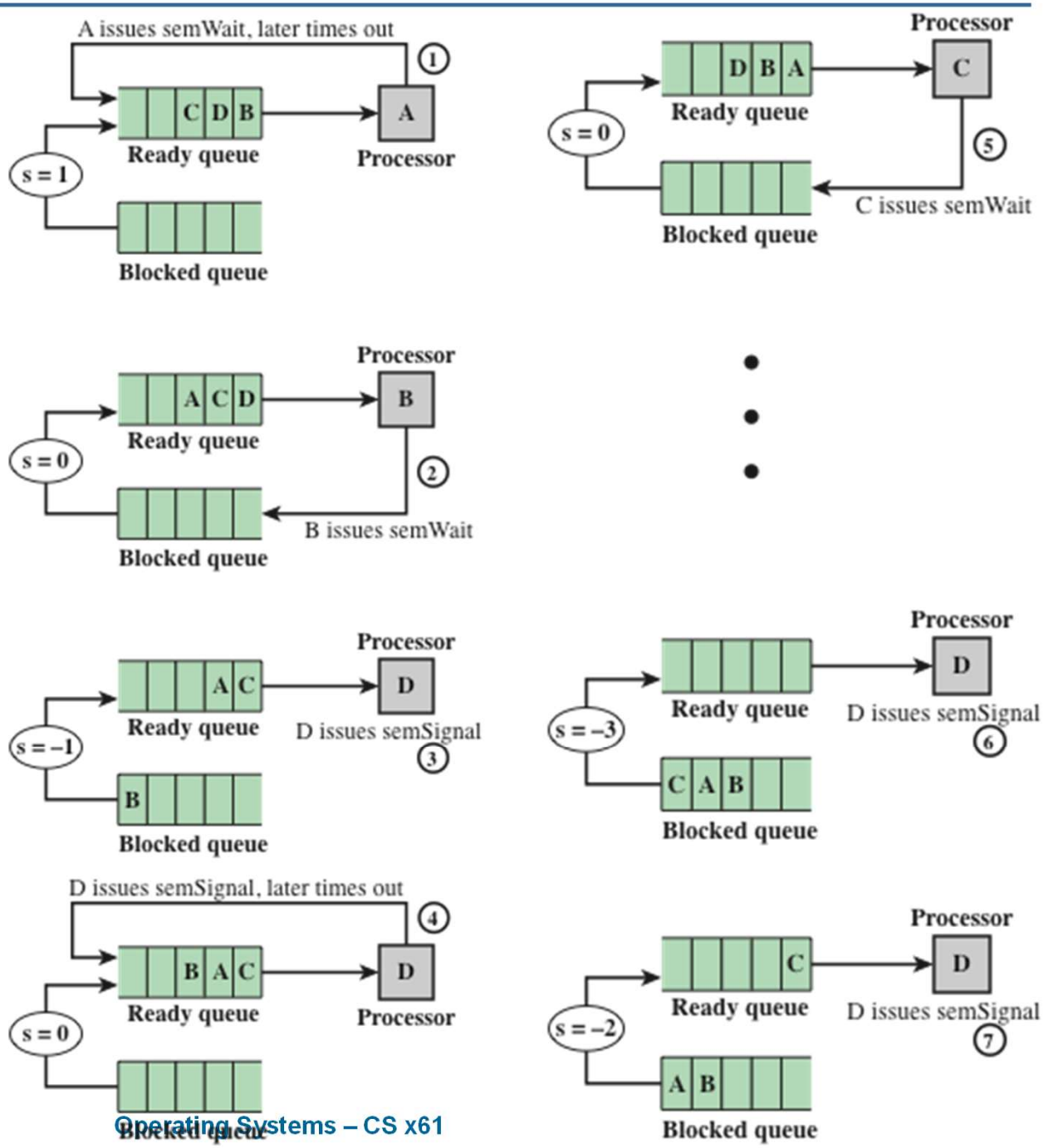
Figure 5.7 Processes Accessing Shared Data Protected by a Semaphore

Strong/Weak Semaphore

- A queue is used to hold processes waiting on the semaphore (for Counting and Binary semaphores)
  - In what order are processes removed from the queue?
- ***Strong Semaphores*** use FIFO
  - The process that has been blocked the longest is released from the queue first
- ***Weak Semaphores*** don't specify the order of removal from the queue
- Strong semaphores guarantee freedom from starvation, while weak semaphores do not

Example of Strong Semaphore Mechanism

Processes A, B, and C depend on a result from process D



Counting Semaphores

- Can support more than one process to be allowed in its critical section at a time
- Initialize the semaphore to the specified value
- s.count is interpreted as:
  - *s.count > 0: s.count is the number of processes that can execute semWait(s)* without suspension (if no semSignal(s) is executed in the meantime).
  - *s.count < 0 :The magnitude of s.count is the number of processes blocked on the semaphore - queue*

```
/* program mutualexclusion */
const int n = /* number of processes  */;
semaphore s = 1;
void P(int i)
{
     while (true) {
          semWait(s);
          /* critical section   */;
          semSignal(s);
          /* remainder   */;
     }
}
void main()
{
     parbegin (P(1), P(2), . . ., P(n));
}
```

Producer/Consumer Problem

- General Scenario:
  - One or more producers are generating data and placing these in a buffer (e.g. records, characters,..)
  - A single consumer is taking items out of the buffer one at time
- Problem statement:
  - Only one agent producer or consumer may access the buffer at any one time
  - Ensure that the Producer can not add data into full buffer
  - Ensure that Consumer can not remove data from empty buffer

Producer/Consumer Problem

- Assume the **buffer is infinite** and consists of linear array of elements, we can define the functions as:

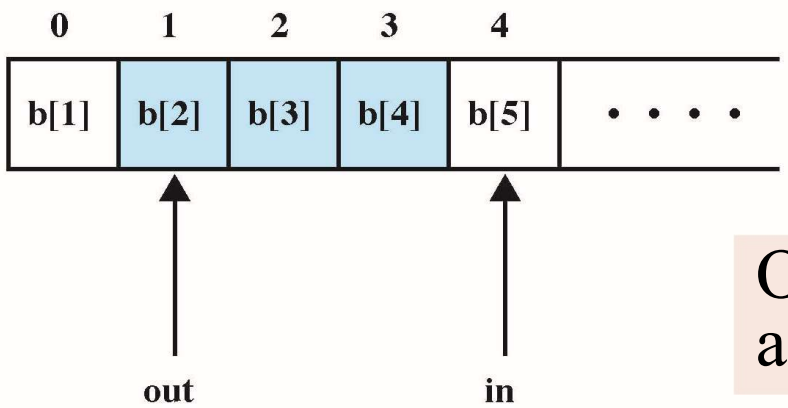
Producer:

```
while (true) {
  /* produce item v */
  b[in] = v;
  in++;
}
```

Consumer:

```
while (true) {
 while (in <= out) /* wait */;
 w = b[out];
 out++;
/* consume item w */
}
```

← ← **Infinite Buffer**



Since ***Buffer*** **b[],** "**in**" and "**out**" pointers are all **shared**, these solutions **do not work**!

Only one ***producer*** or ***consumer*** should access the buffer at any one time !

Implementation using Binary Semaphores

```
/* program producerconsumer */
int n;
binary_semaphore s = 1, delay = 0;
void producer()
{
     while (true) {
          produce();
          semWaitB(s);
          append();
          n++;
          if (n==1) semSignalB(delay);
          semSignalB(s);
     }
}
void consumer()
{
     semWaitB(delay);
     while (true) {
          semWaitB(s);
          take();
          n--;
          semSignalB(s);
          consume();
          if (n==0) semWaitB(delay);
     }
}
void main()
{
     n = 0;
     parbegin (producer, consumer);
}
```

n # items in buffer
= in - out

s : enforce mutual exclusion

delay: force consumer to semWait if the buffer is empty

There is a flaw in this program!

Possible Scenario

	Producer	Consumer	s	n	Delay
1			1	0	0
2	semWaitB(s)		0	0	0
3	n++		0	1	0
4	**if** (n==1) (semSignalB(delay))		0	1	1
5	semSignalB(s)		1	1	1
6		semWaitB(delay)	1	1	0
7		semWaitB(s)	0	1	0
8		n--	0	0	0
9		semSignalB(s)	1	0	0
10	semWaitB(s)		0	0	0
11	n++		0	1	0
12	**if** (n==1) (semSignalB(delay))		0	1	1
13	semSignalB(s)		1	1	1
14		**if** (n==0) (semWaitB(delay))	1	1	1
15		semWaitB(s)	0	1	1
16		n--	0	0	1
17		semSignalB(s)	1	0	1
18		**if** (n==0) (semWaitB(delay))	1	0	0
19		semWaitB(s)	0	0	0
20		n--	0	–1	0
21		semiSignlaB(s)	1	–1	0

NOTE: White areas represent the critical section controlled by semaphore s.

```
/* program producerconsumer */
int n;
binary_semaphore s = 1, delay = 0;
void producer()
{
     while (true) {
          produce();
          semWaitB(s);
          append();
          n++;
          if (n==1) semSignalB(delay);
          semSignalB(s);
     }
}
void consumer()
{
     int m; /* a local variable */
     semWaitB(delay);
     while (true)  {
          semWaitB(s);
          take();
          n--;
          m = n;
          semSignalB(s);
          consume();
          if (m==0) semWaitB(delay);
     }
}
void main()
{
     n = 0;
     parbegin (producer, consumer);
}
```

Fix: introduce an auxiliary variable m that can be set in the consumer's critical section for use later on

More Elegant Solution

```
/* program  producerconsumer */
semaphore n = 0, s = 1;
void producer()
{
     while (true) {
          produce();
          semWait(s);
          append();
          semSignal(s);
          semSignal(n);
     }
}
void consumer()
{
     while (true) {
          semWait(n);
          semWait(s);
          take();
          semSignal(s);
          consume();
     }
}
void main()
{
     parbegin (producer, consumer);
}
```

Let n (# items in buffer) be a Counting Semaphore

Q: if *semSignal(s)* & *semSignal(n)* are reversed, would it lead to deadlock?

A: No because consumer has to wait on both semaphores before proceeding

Q: if *semWait(n)* & *semWait(s)* are reversed, would it lead to deadlock?

A: Yes because if consumer enters its critical section when the buffer is empty, then no producer can ever append to the buffer and the system is deadlocked.

Bounded Buffer

Block on:	Unblock on:
Producer: insert in full buffer	Consumer: item inserted
Consumer: remove from empty buffer	Producer: item removed

buffer is treated as a circular storage

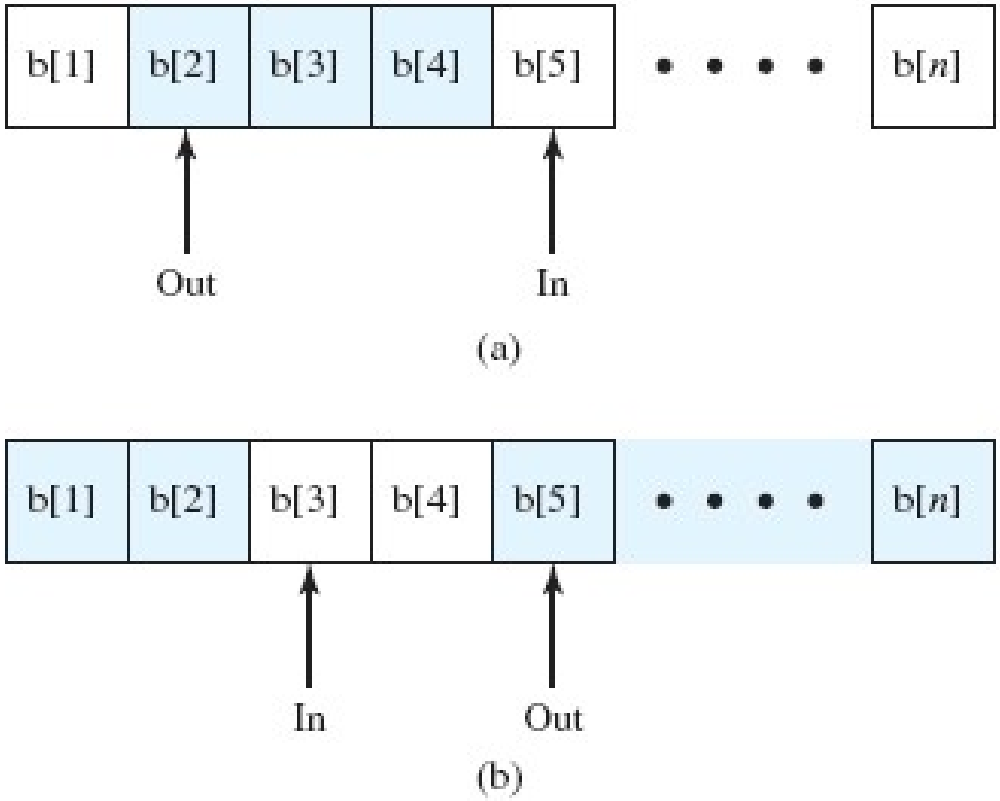
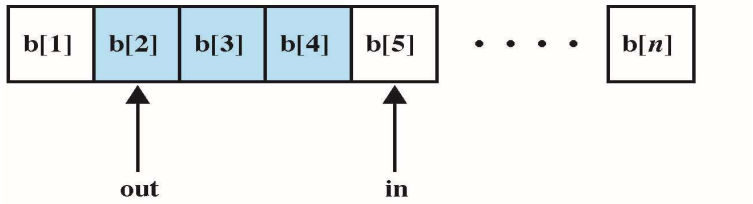


Figure 5.12 Finite Circular Buffer for the Producer/Consumer Problem

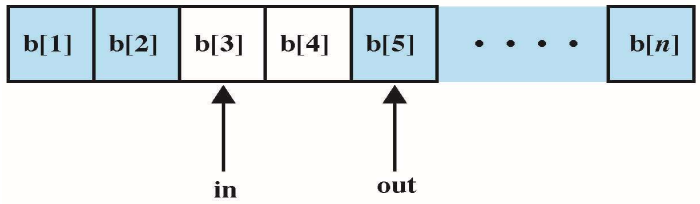
Producer/Consumer using a Circular Buffer

```
Producer:
while (true) {

  /* produce item v */

  while((in + 1)%n==out)
        /* do nothing */;

  b[in] = v;
  in = (in + 1) % n
}
```

```
Consumer:
while (true) {

   while (in == out)
      /* do nothing */;

   w = b[out];
   out = (out + 1) % n;

   /* consume item w */
}
```



(a)



(b)

Since ***Buffer*** **b[]**, "**in**" and "**out**" pointers are all **shared**, we have to enforce mutual exclusion

Bounded-Buffer Problem

- ***n*** buffers, each can hold one item
- Semaphore **mutex** initialized to the value 1
- Semaphore **full** initialized to 0 – counting number of full buffers
- Semaphore **empty** initialized to n – counting number of empty buffers

- **Producer**

```
do {
    ...
  /* produce an item in
  next_produced */
    ...
   wait(empty);
   wait(mutex);
    ...
  /* add next produced to
  the buffer */
            ...
    signal(mutex);
    signal(full);
  } while (true);
```

- **Consumer**

```
do {
    wait(full);
    wait(mutex);
    ...
    /* remove an item from
buffer to next_consumed */
    ...
    signal(mutex);
    signal(empty);
    ...
    /* consume the item in
next consumed */
            ...
} while (true);
```

Readers-Writers Problem

- A data area is shared among many processes (e.g. file, block of memory, …)
- Some processes only read the data area: readers
- Some processes only write to the data area: writers
- **Conditions** to satisfy:
  1. Multiple readers may read the file simultaneously
  2. Only one writer at a time may write
  3. If a writer is writing to the file, no reader may read it.
- Therefore:
  - Readers are not required to exclude one another
  - Writers are processes required to exclude all other processes whether readers or writers

Readers/Writers Problem

- Can we apply the general mutual exclusion to solve the R/W problem? declare any portion of a process that accesses the data area to be a critical section and impose the general mutual exclusion solution
  - inefficient and unacceptably slow as users are forced to read one at a time
- Can we consider the producer/consumer problem a special case of R/W problem with a single writer (the producer) and a single reader (the consumer)?
  - The answer is **no**.
  - The producer is not just a writer. It must read queue pointers to determine where to write the next item, and it must determine if the buffer is full.
  - Similarly, the consumer is not just a reader, because it must adjust the queue pointers to show that it has removed a unit from the buffer.

Semaphores - Readers have Priority

- Writer and Reader use **wsem** to enforce mutual exclusion
- To allow multiple Readers
  - First Reader wait on wsem
  - > one reader, subsequent readers need not wait
  - **readcount** keep track of # Readers
  - Semaphore **x** protect global variable readcount
- Once a single reader access data area, it is possible to retain control as long as there is at least one reader reading

→ writers are subject to starvation

```
/* program readersandwriters */
int readcount;
semaphore x = 1, wsem = 1;
void reader()
{
    while (true) {
      semWait (x);
      readcount++;
      if (readcount == 1) semWait (wsem);
      semSignal (x);
      READUNIT();
      semWait (x);
      readcount--;
      if (readcount == 0) semSignal (wsem);
      semSignal (x);
    }
 }
void writer()
{
    while (true) {
      semWait (wsem);
      WRITEUNIT();
      semSignal (wsem);
    }
}

void main()
{
    readcount = 0;
    parbegin (reader, writer);
}
```

Semaphores- Writers have Priority

- No new readers are allowed access to the data area once at least one writer has desire to write
- For **writers**, add:
  - A semaphore **rsem**: inhibits all readers while there is at least one writer desiring access to the data area
  - variable **writecount** controls the setting of rsem
  - A semaphore **y** that controls the updating of writecount

```
void writer ()
{
    while (true) {
      semWait (y);
           writecount++;
           if (writecount == 1) semWait (rsem);
      semSignal (y);
      semWait (wsem);
      WRITEUNIT();
      semSignal (wsem);
      semWait (y);
           writecount--;
           if (writecount == 0) semSignal (rsem);
      semSignal (y);
    }
}
void main()
{
    readcount = writecount = 0;
    parbegin (reader, writer);
}
```

Semaphores- Writers have Priority (Contd)

- For **readers**, one additional semaphore is needed:
- A long queue must not be allowed to build up on rsem from Readers; otherwise writers will not be able to jump the queue.
- Therefore, only one reader is allowed to queue on rsem, with any additional readers queuing on semaphore **z**, immediately before waiting on rsem.

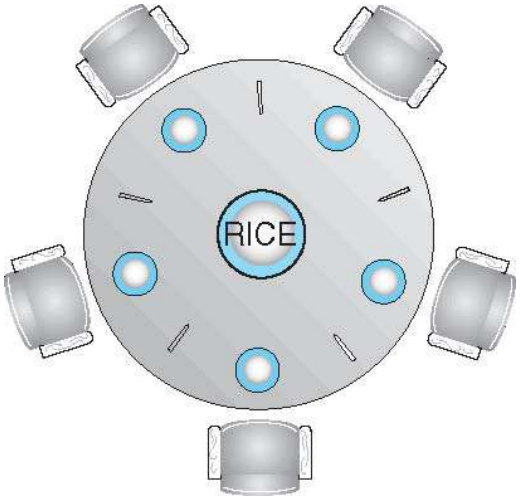
```
/* program readersandwriters */
int  readcount, writecount;
semaphore x = 1, y = 1, z = 1, wsem = 1, rsem = 1;
void reader()
{
    while (true) {
      semWait (z);
           semWait (rsem);
                semWait (x);
                     readcount++;
                     if (readcount == 1) semWait (wsem);
                semSignal (x);
           semSignal (rsem);
      semSignal (z);
      READUNIT();
      semWait (x);
           readcount--;
           if (readcount == 0) semSignal (wsem);
      semSignal (x);
    }
}
```

Semaphores- Writers have Priority (Contd)

Table 5.6 State of the Process Queues for Program of Figure 5.23

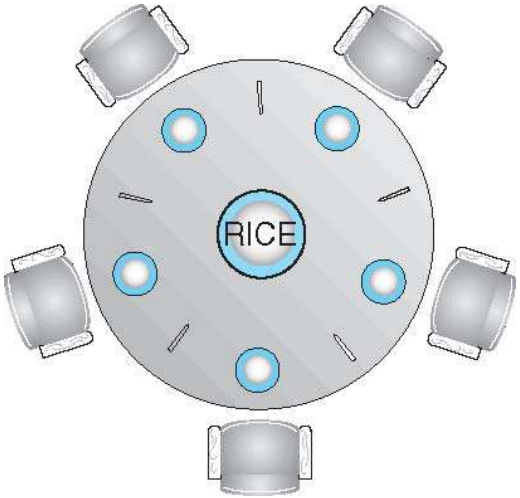
Readers only in the system	• *wsem* set • no queues
Writers only in the system	• *wsem* and *rsem* set • writers queue on *wsem*
Both readers and writers with read first	• *wsem* set by reader • *rsem* set by writer • all writers queue on *wsem* • one reader queues on *rsem* • other readers queue on *z*
Both readers and writers with write first	• *wsem* set by writer • *rsem* set by writer • writers queue on *wsem* • one reader queues on *rsem* • other readers queue on *z*

Dining Philosophers Problem



- Five philosophers live in a house
- The life of each philosopher consists of thinking and eating
- each philosopher requires two chopsticks to eat rice
- Eating arrangements: a round table with a bowl of rice, five plates, one for each philosopher, and five chopsticks.
- A philosopher wishing to eat try to pick up 2 chopsticks (one at a time) to eat from bowl
  - Need both to eat, then release both when done

Dining Philosophers Problem



- Devise a ritual (algorithm) that will allow the philosophers to eat.
  - No two philosophers can use the same chopstick at the same time (mutual exclusion)
  - No philosopher must starve to death (avoid deadlock and starvation)
- One of the classic problems
  - Deadlock and starvation
  - Coordination of shared resources

Dining Philosophers Solution

- Inform the philosophers to behave as follows
  - think until the left fork is available; when it is, pick it up
  - think until the right fork is available; when it is, pick it up
  - eat
  - put the left fork down
  - put the right fork down
  - repeat from the start

Dining-Philosophers Problem Algorithm

- In the case of 5 philosophers, the Shared data
  - Bowl of rice (data set)
  - Semaphore **chopstick [5]** initialized to 1
- The structure of Philosopher *i*:

```
do {
        wait (chopstick[i] );
        wait (chopStick[(i+1)%5]);
                    //  eat
        signal (chopstick[i]);
        signal (chopstick[(i+1)%5]);
                    //  think
} while (TRUE);
```

- What is the problem with this algorithm?

If all of the philosophers are hungry at the same time, they all pick up the chopstick on their left. All the elements of chopstick will now be equal to 0. When each philosopher tries to grab her right chopstick, she will be delayed forever. Deadlock and Starvation

Avoiding deadlock

- add an attendant who only allows four philosophers at a time in dining room.
- With max four seated philosophers, at least one will have access to two forks.

```
/* program diningphilosophers */
semaphore fork[5] = {1};
semaphore room = {4};
int i;
void philosopher (int i)
{
    while (true) {
     think();
     wait (room);
     wait (fork[i]);
     wait (fork [(i+1) mod 5]);
     eat();
     signal (fork [(i+1) mod 5]);
     signal (fork[i]);
     signal (room);
    }

}
void main()
{
    parbegin (philosopher (0), philosopher (1), philosopher (2),
          philosopher (3), philosopher (4));
}
```

Problems with Semaphores

- Incorrect use of semaphore operations can result in timing errors that are difficult to detect as these sequences do not always occur
  - E.g. counter problem in Producer-Consumer problem
- If a process interchanges order of signal() and wait()
  - signal (mutex) ... critical section…. wait (mutex)
  - several processes maybe executing in their critical sections simultaneously, violating the mutual-exclusion requirement
- If a process replaces a signal() with wait()
  - wait (mutex) ... critical section … wait (mutex)
  - Deadlock will occur
- If a process omits wait (mutex) or signal (mutex) (or both)
  - either mutual exclusion is violated or a deadlock will occur
- Deadlock and starvation are possible.
- researchers have developed high-level language constructs: **Monitors**

Monitors

- A high-level abstraction that provides a convenient and effective mechanism for process synchronization
- Monitor is a programming-language construct that provides equivalent functionality to semaphores and easier to control
  - But not powerful enough to model some synchronization schemes
- *A* ***monitor type*** *is an* **Abstract Data Type (ADT)**, that includes a set of programmer defined operations that are provided with mutual exclusion within the monitor
- Implemented in some programming languages:
  - Concurrent Pascal, Pascal-Plus, Modula-2, Modula-3, C# and Java

Monitors

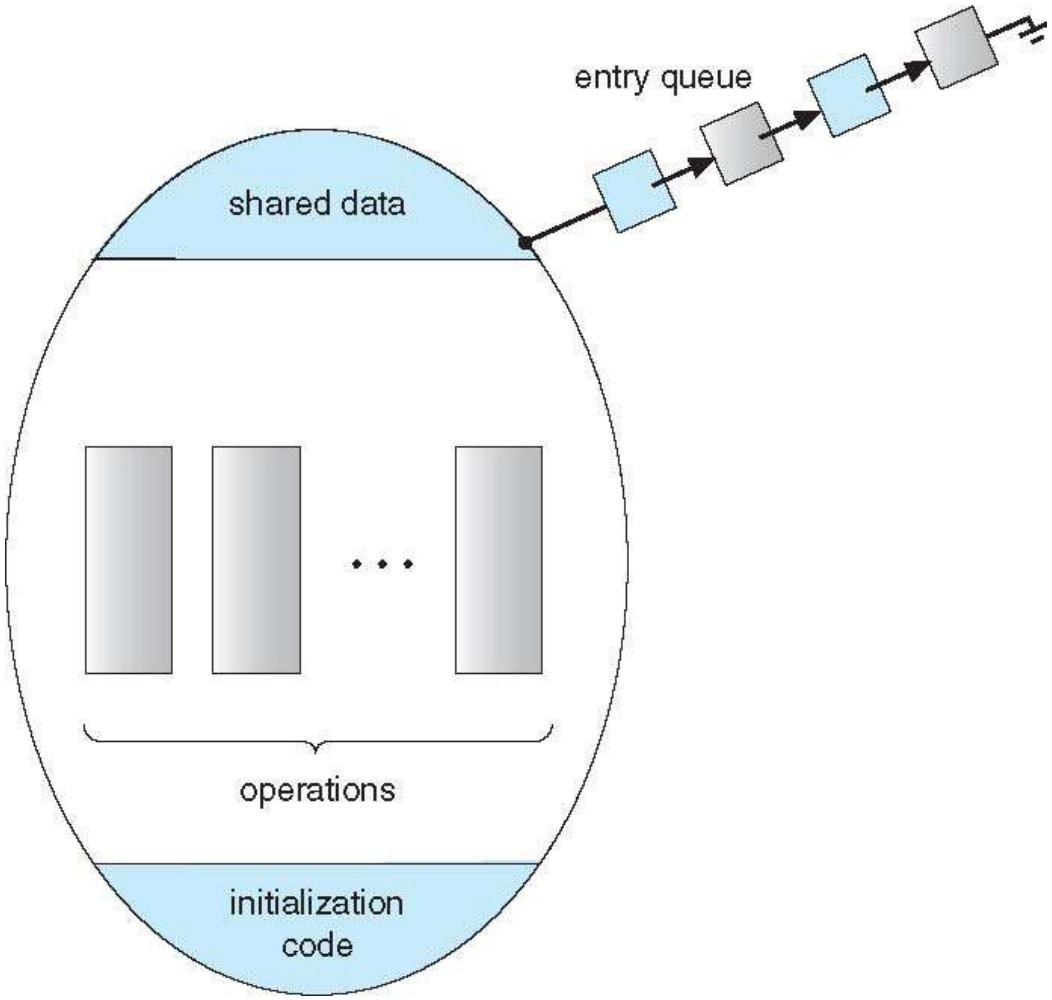
- Monitor is a software module with
  - local data
  - initialisation section
  - One ore more procedures

```
monitor monitor-name
{
   //shared variable declarations
    procedure P1 (…) { …. }

    procedure Pn (…) {……}

    Initialization code (…) { …
      }
}
```

- Monitor modules have the following characteristics
  - Local data variables are accessible only by the monitor, not by any external procedures (OO)
  - A process enters monitor by invoking one of its procedures (OO)
  - *Only one process may be executing in the monitor at a time; any other processes that have invoked the monitor are blocked, waiting for the monitor to become available*

Schematic view of a Monitor

- The monitor construct ensures that only one process at a time is active within the monitor.
- Consequently, the programmer does not need to code this synchronization constraint explicitly



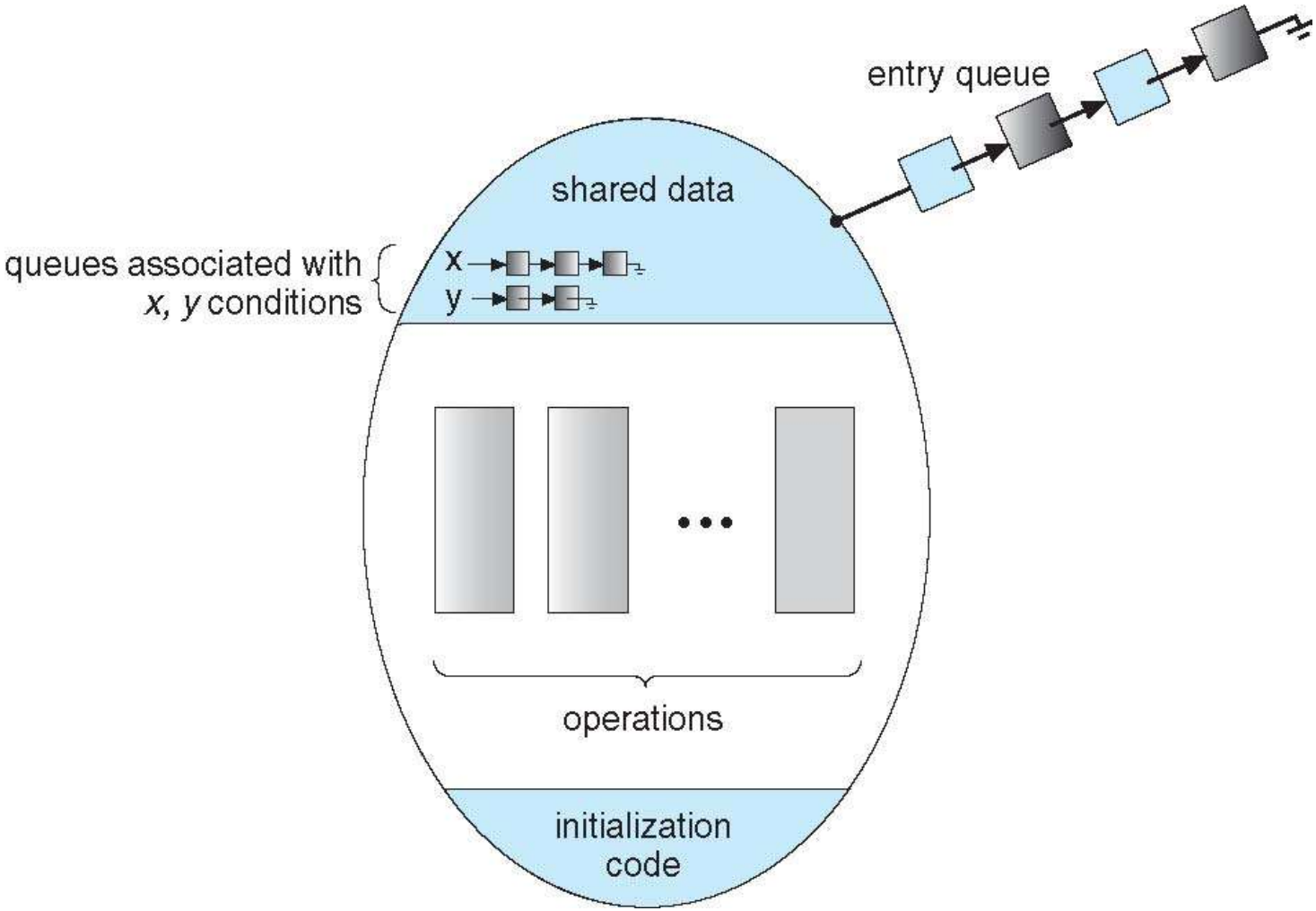
Monitors

- By enforcing the discipline of one process at a time, the monitor is able to provide a mutual exclusion facility
  - a shared data structure can be protected by placing it in a monitor
  - If the data in a monitor represent a resource, then the monitor provides a mutual exclusion facility for accessing the resource
- To be useful for concurrent processing, the monitor must include **synchronization tools**
  - For example, suppose a process invokes the monitor and, while in the monitor, must be blocked until some condition is satisfied. A facility is needed by which the process is not only blocked but releases the monitor so that some other process may enter it. Later, when the condition is satisfied and the monitor is again available, the process needs to be resumed and allowed to reenter the monitor at the point of its suspension

Condition Variables

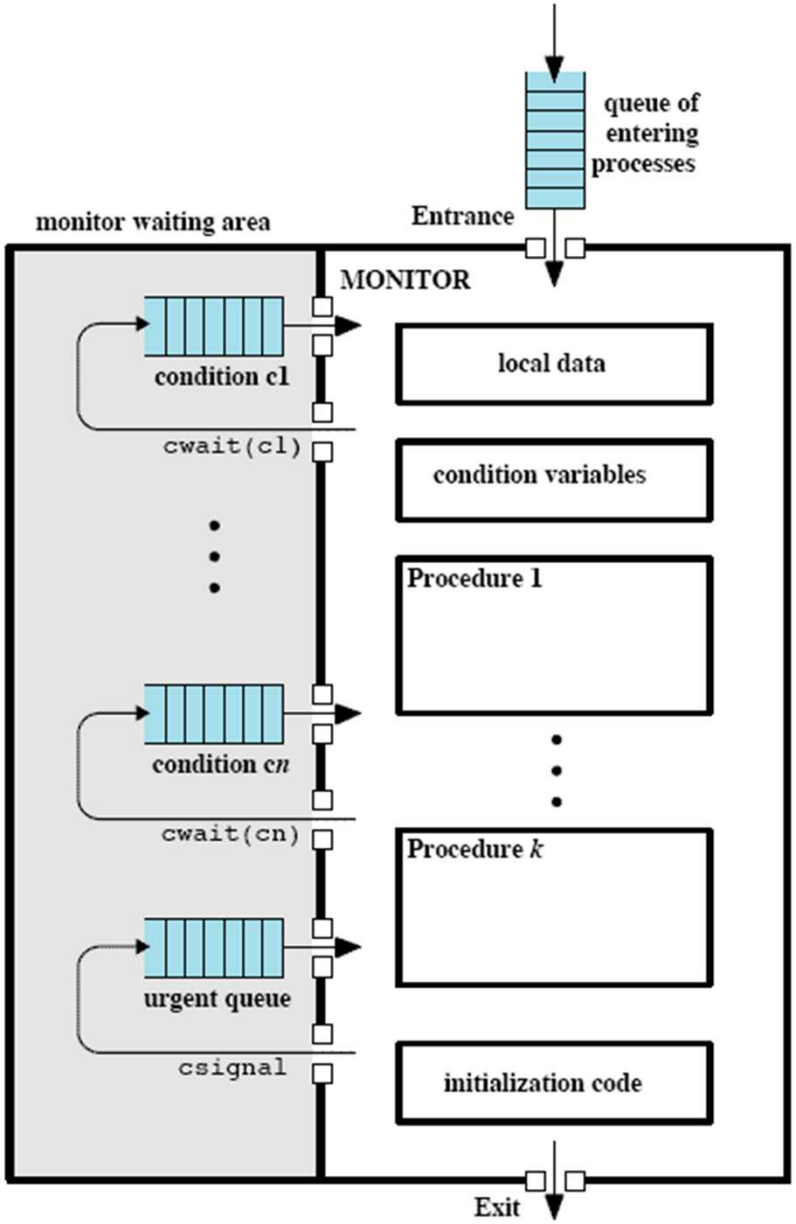
- Synchronisation with a monitor is achieved by **condition variables**
  - special data type in monitors
  - only accessible by the monitor
- `condition x, y;`
- Two operations are allowed on a condition variable:
  - `x.wait()` – a process that invokes the operation is suspended until `x.signal()` –
  - `x.signal()` – resumes one of processes (if any) that invoked `x.wait()`
    - If no `x.wait()` on the variable, then it has no effect on the variable
- Note that monitor wait and signal operations are different from those for the semaphore. ***If a process in a monitor signals and no task is waiting on the condition variable, the signal is lost***.

Monitor with Condition Variables



Structure of a Monitor

- A monitor has a single entry point: it is guarded such that only one process may be in the monitor at a time
- Other processes attempting to enter the monitor join a queue waiting for the monitor
- Once a process is in the monitor, it may temporarily block itself on condition x by issuing x.wait();
  - it is then placed in a condition queue of processes waiting to re-enter the monitor when the condition changes, and resume execution at the point in its program following the x.wait() call.
- when a process executing in the monitor issues x.signal(), it alerts the corresponding condition queue that the condition has changed.



Condition Variables Choices

- If process P invokes `x.signal()`, and process Q is suspended in `x.wait()`, what should happen next?
  - Both Q and P **cannot execute in parallel**: only one process active in the monitor at a time - If Q is resumed, then P must wait
- **Options** include
  - **Signal and wait** – P waits until Q either leaves the monitor or it waits for another condition
  - **Signal and continue** – Q waits until P either leaves the monitor or it waits for another condition
  - Both have pros and cons – language implementer can decide
  - Monitors implemented in Concurrent Pascal compromise
    - P executing signal() immediately leaves the monitor, Q is resumed
  - Implemented in other languages including Mesa, C#, Java

Bounded Buffer Solution Using Monitor

- A producer can add characters to the buffer only by means of the procedure append(x) inside the monitor
- the producer does not have direct access to *buffer*
- A consumer can obtain characters from the buffer only thru take(x) inside a monitor

```
void producer()
{
     char x;
     while (true) {
     produce(x);
     append(x);
     }
}
void consumer()
{
     char x;
     while (true) {
       take(x);
       consume(x);
     }
}
void main()
{
     parbegin (producer, consumer);
}
```

Bounded Buffer Solution Using Monitor

```
/* program producerconsumer */
monitor boundedbuffer;
char buffer [N];                                    /* space for N items */
int nextin, nextout;                                  /* buffer pointers */
int count;                                     /* number of items in buffer */
cond notfull, notempty;          /* condition variables for synchronization */

void append (char x)
{
     if (count == N) cwait(notfull);       /* buffer is full; avoid overflow */
     buffer[nextin] = x;
     nextin = (nextin + 1) % N;
     count++;
     /* one more item in buffer */
     csignal(notempty);                        /* resume any waiting consumer */
}
void take (char x)
{
     if (count == 0) cwait(notempty);   /* buffer is empty; avoid underflow */
     x = buffer[nextout];
     nextout = (nextout + 1) % N;
     count--;                                   /* one fewer item in buffer */
     csignal(notfull);                         /* resume any waiting producer */
}
{                                                          /* monitor body */
     nextin = 0; nextout = 0; count = 0;            /* buffer initially empty */
}
```

Monitor Solution to Dining Philosophers

```
monitor DiningPhilosophers {
enum {THINKING,HUNGRY,EATING)
   state[5];
condition self [5];

void pickup (int i) {
    state[i] = HUNGRY;
    test(i);
    if (state[i] != EATING)
        self[i].wait;
}

void putdown (int i) {
   state[i] = THINKING;
// test left and right neighbors
   test((i+4) % 5);
    test((i+1) % 5);
}
```

```
void test (int i) {
   if ((state[(i+4)%5] != EATING) &&
        (state[i] == HUNGRY) &&
        (state[(i+1)%5] != EATING)) {
            state[i] = EATING ;
            self[i].signal() ;
   }
 }

initialization_code() {
   for (int i = 0; i < 5; i++)
      state[i] = THINKING;
  }
}
```

Solution to Dining Philosophers (Cont.)

- Each philosopher *i* invokes the operations **`pickup()`** and **`putdown()`** in the following sequence:

```
DiningPhilosophers.pickup(i);

EAT

DiningPhilosophers.putdown(i);
```

- this solution ensures that no two neighbors are eating simultaneously-- No deadlock
- but starvation is possible

Monitor Implementation Using Semaphores

- Variables

```
semaphore mutex;   // (initially = 1)
semaphore next;    // (initially = 0)
int next_count = 0; //#processes suspended on next
```

- A signaling process must wait until the resumed process either leaves or waits – the process can use **`next`** to **suspend itself**

- Each procedure ***F*** will be replaced by

```
wait(mutex);
      …
      body of F;
      …
if (next_count > 0)
  signal(next)
else
  signal(mutex);
```

- Mutual exclusion within a monitor is ensured

Monitor Implementation – Condition Variables

- For each condition variable ***x***, we have:

```
semaphore x_sem; // (initially  = 0)
int x_count = 0;
```

- The operation **x.wait** can be implemented as:

```
x_count++;
if (next_count > 0)
    signal(next);
else
    signal(mutex);
wait(x_sem);
x_count--;
```

- The operation **x.signal** can be implemented as:

```
if (x_count > 0) {
    next_count++;
    signal(x_sem);
    wait(next);
    next_count--;
}
```

Process Interaction

- When processes interact with one another, two fundamental requirements must be satisfied:
  - communication: cooperating processes need to exchange information
  - synchronization: Processes need to be synchronized to enforce mutual exclusion
- Message Passing provides both functions
  - Added bonus: It works with shared memory *and* with distributed systems

Message Passing

- The actual function of message passing is normally provided in the form of a pair of primitives:
  - send (destination, message)
    - A process sends information in the form of a message to another process designated by a destination
  - receive (source, message)
    - A process receives information by executing the receive primitive, indicating the source and the message
- Crucial for synchronization: a message can be consumed only once

Synchronization

- Communication requires synchronization
  - Sender must send before receiver can receive
- What happens to a process after it issues a send or receive primitive?
  - Sender is either
    - Blocked until the message is received
    - Not blocked and continue execution
  - Receiver
    - If a message has previously been sent, the message is received and execution continues
    - If there is no waiting message, then either

      (a) the process is blocked until a message arrives, or

      (b) the process continues to execute, abandoning the attempt to receive

Blocking send / Blocking receive

- Both sender and receiver are blocked until message is delivered
- Known as a *rendez-vous*
- Allows for tight synchronization between processes.

Non-blocking Send

- More natural for many concurrent programming tasks
  - send a request message (e.g. print) and carry on
- Non-blocking send, nonblocking receive
  - Neither party is required to wait
- Non-blocking send, blocking receive
  - Sender continues on
  - Receiver is blocked until the requested message arrives
  - Most useful combination:
    - It allows a process to send one or more messages to a variety of destinations as quickly as possible
    - A process that must receive a message before continuing certain task is blocked until such a message arrives

Addressing

- Sending process need to be able to specify which process should receive the message
  - Direct addressing
  - Indirect Addressing

Direct Addressing

- Send primitive includes a specific identifier of the destination process
- Receive primitive, two ways
  - the receiving process explicitly designate a sending process
    - But process must know ahead of time from which process a message is expected
    - Effective for cooperating concurrent processes
    - Not the case for many applications, e.g. a print server can accept requests from any process
  - Implicit addressing: Receive primitive could use source parameter to return a value when the receive operation has been performed

Indirect addressing

- Messages are sent to a shared data structure, queues, that holds messages temporarily
- Queues are called *mailboxes*
- One process sends a message to the mailbox and the other process picks up the message from the mailbox
- Strength: by decoupling the sender and receiver, it allows for greater flexibility in the use of messages

IPC: Relationship between Senders and Receivers

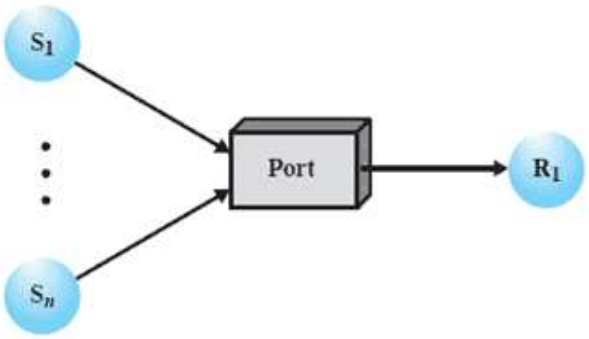
One-to-One relationship

- allows a private communications link to be set up between two processes
- insulates their interaction from erroneous interference from other processes

(a) One to one

Many-to-One relationship

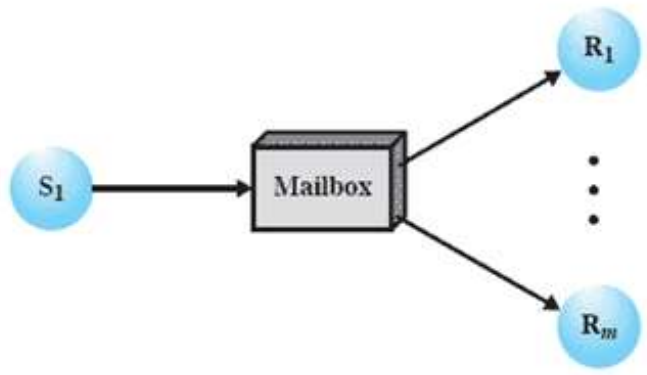
- one process provides service to a number of other processes
- In this case, the mailbox is often referred to as a port
- useful for client/server interaction



(b) Many to one

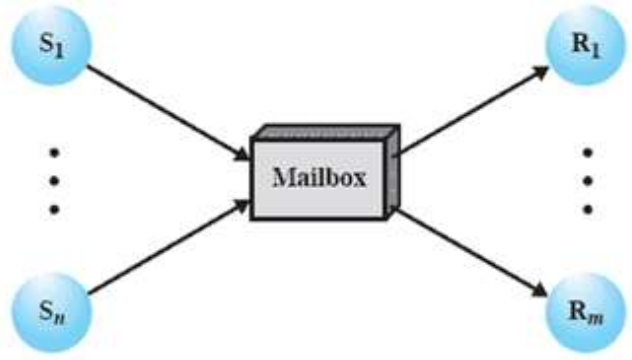
IPC: Relationship between Senders and Receivers

- **One-to-Many relationship**
  - allows for one sender and multiple receivers;
  - useful for applications where a message is to be broadcast to a set of processes



(c) One to many

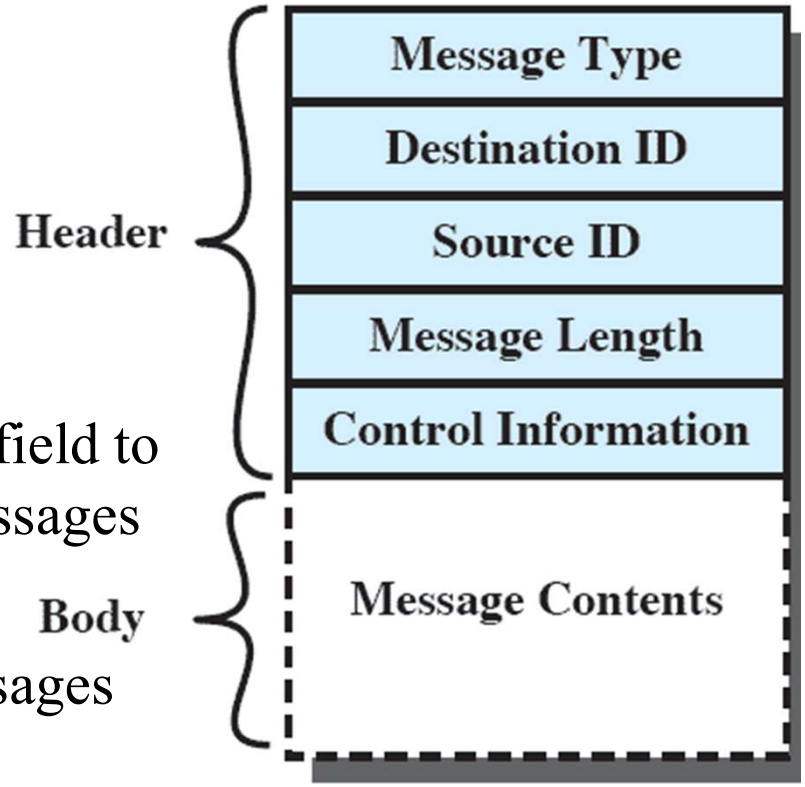
- **Many-to-Many relationship**
  - allows multiple server processes to provide concurrent service to multiple clients



(d) Many to many

General Message Format

- Depends on objectives of messaging facility
  - Short, fixed-length messages to minimize processing and storage overhead
  - Variable-length messages to allow flexibility
- Variable length messages
  - Header contains
    - message information e.g. Source id, destination id, a length field, and a type field to discriminate among various types of messages
    - control information
      - pointer field so a linked list of messages can be created
      - a sequence number to keep track of order
      - Priority, etc...
  - Body: contains contents of the message.



Mutual Exclusion Using Messages (NB send, B receive)

- A set of concurrent processes share a mailbox, box, initialized with a single message
- For a process to enter its critical section it attempts to receive a message
- If the mailbox is empty, then the process is blocked
- Once a process has acquired the message, it performs its critical section and then places the message back into the mailbox.
- Thus, the message functions as a token that is passed from process to process

```
/* program mutualexclusion */
const int n = /* number of processes  */;
void P(int i)
{
    message msg;
    while (true) {
     receive (box, msg);
     /* critical section   */;
     send (box, msg);
     /* remainder   */;
    }
}
void main()
{
    create mailbox (box);
    send (box, null);
    parbegin (P(1), P(2), . . ., P(n));
}
```

Mutual Exclusion Using Messages (NB send, B receive)

- This assumes that if more than one process performs the receive operation concurrently, then
  - If there is a message, it is delivered to only one process and the others are blocked, or
  - If the message queue is empty, all processes are blocked; when a message is available, only one blocked process is activated and given the message.

```
/* program mutualexclusion */
const int n = /* number of processes  */;
void P(int i)
{
    message msg;
    while (true) {
     receive (box, msg);
     /* critical section   */;
     send (box, msg);
     /* remainder   */;
    }
}
void main()
{
    create mailbox (box);
    send (box, null);
    parbegin (P(1), P(2), . . ., P(n));
}
```

Bounded-Buffer Producer/Consumer using Messages

```
const int
    capacity = /* buffering capacity */ ;
    null =/* empty message */ ;
int i;
void producer()
{   message pmsg;
    while (true) {
     receive (mayproduce, pmsg);
     pmsg = produce();
     send (mayconsume, pmsg);
    }
}
void consumer()
{   message cmsg;
    while (true) {
     receive (mayconsume, cmsg);
     consume (cmsg);
     send (mayproduce, null);
    }
}

void main()
{
    create_mailbox (mayproduce);
    create_mailbox (mayconsume);
    for (int i = 1; i <= capacity; i++) send (mayproduce, null);
    parbegin (producer, consumer);
}
```

Message Passing Writers have Priority

```
void  controller()
{
      while (true)
      {
         if (count > 0) {
            if (!empty (finished)) {
               receive (finished, msg);
               count++;
            }
            else if (!empty (writerequest)) {
               receive (writerequest, msg);
               writer_id = msg.id;
               count = count – 100;
            }
            else if (!empty (readrequest)) {
               receive (readrequest, msg);
               count--;
               send (msg.id, "OK");
            }
         }
         if (count == 0) {
            send (writer id, "OK");
            receive (finished, msg);
            count = 100;
         }
         while (count < 0) {
            receive (finished, msg);
            count++;
         }
      }
}
```

```
void reader(int i)
{
   message rmsg;
      while (true) {
         rmsg = i;
         send (readrequest, rmsg);
         receive (mbox[i], rmsg);
         READUNIT ();
         rmsg = i;
         send (finished, rmsg);
      }
 }
void writer(int j)
{
   message rmsg;
   while(true) {
      rmsg = j;
      send (writerequest, rmsg);
      receive (mbox[j], rmsg);
      WRITEUNIT ();
      rmsg = j;
      send (finished, rmsg);
   }
}
```

Message Passing Writers have Priority

- there is a **controller process** that has access to the shared data area
- **Readers** and **Writers** processes wishing to **access** the data area
  - **send request message** to controller
  - are **granted access** with an **OK reply** message
  - **indicate completion** of access with **a "finished" message**
- The controller is equipped with **three mailboxes**, one for each type of message that it may receive

1. Readrequest
2. Writerequest
3. Finished

```
void reader(int i)
{
   message rmsg;
      while (true) {
         rmsg = i;
         send (readrequest, rmsg);
         receive (mbox[i], rmsg);
         READUNIT ();
         rmsg = i;
         send (finished, rmsg);
      }
 }
void writer(int j)
{
   message rmsg;
   while(true) {
      rmsg = j;
      send (writerequest, rmsg);
      receive (mbox[j], rmsg);
      WRITEUNIT ();
      rmsg = j;
      send (finished, rmsg);
   }
}
```

Message Passing Writers have Priority

- Controller must
  - Enforce mutual exclusion
  - services write request messages before read request
- variable *count* is initialized to a number > max # possible readers (here 100)
- If count > 0, then no writer is waiting and there may or may not be readers active. Service all "finished" messages first to clear active readers. Then service write requests and then read requests.
- If count =0, then the only request outstanding is a write request. Allow the writer to proceed and wait for a "finished" message.
- If count < 0, then a writer has made a request and is being made to wait to clear all active readers. Therefore, only "finished" messages should be serviced

```
void  controller()
{
      while (true)
      {
         if (count > 0) {
            if (!empty (finished)) {
               receive (finished, msg);
               count++;
            }
            else if (!empty (writerequest)) {
               receive (writerequest, msg);
               writer_id = msg.id;
               count = count – 100;
            }
            else if (!empty (readrequest)) {
               receive (readrequest, msg);
               count--;
               send (msg.id, "OK");
            }
         }
         if (count == 0) {
            send (writer id, "OK");
            receive (finished, msg);
            count = 100;
         }
         while (count < 0) {
            receive (finished, msg);
            count++;
         }
      }
}
```

Synchronization Examples

- Solaris
- Windows
- Linux
- Pthreads

Windows Synchronization

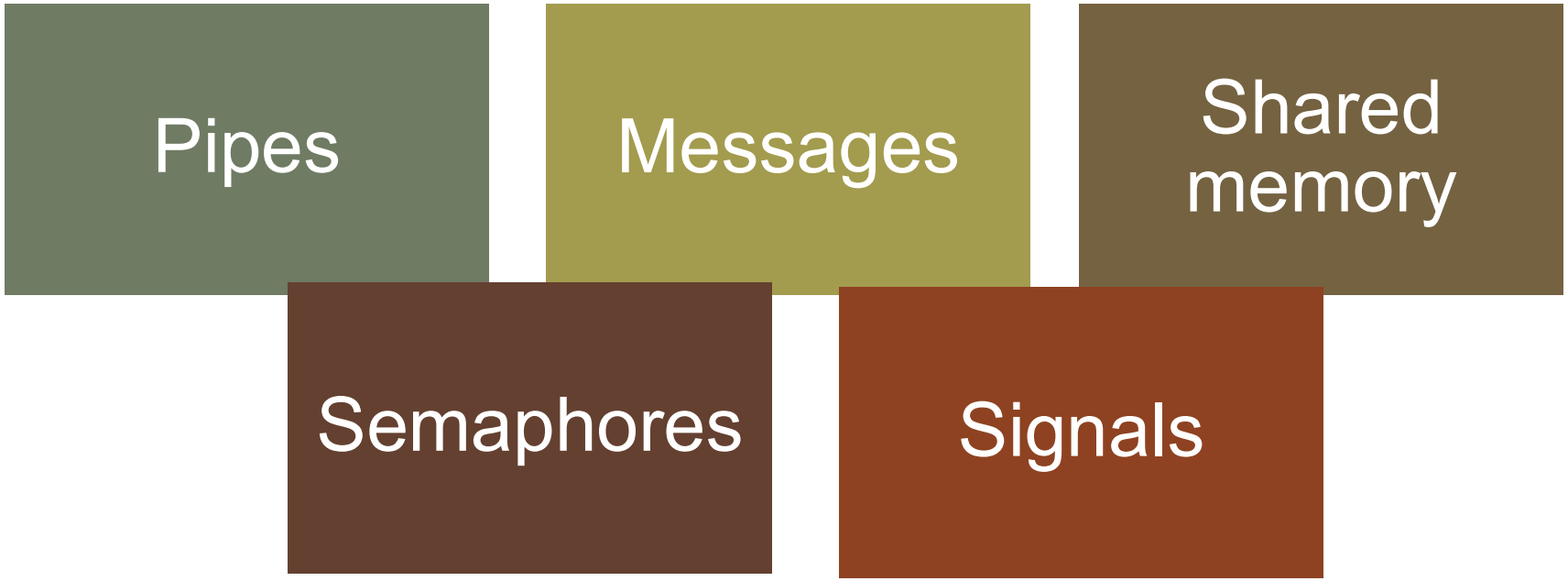
- Uses interrupt masks to protect access to global resources on uniprocessor systems
- Uses **spinlocks** on multiprocessor systems
  - Spinlocking-thread will never be preempted
- Also provides **dispatcher objects** user-land which may act mutexes, semaphores, events, and timers
  - **Events** An event acts much like a condition variable
  - **Timers** notify one or more thread when time expired
  - Dispatcher objects either **signaled-state** (object available) or **non-signaled state** (thread will block)

Linux Synchronization

- Linux:
  - Prior to kernel Version 2.6, disables interrupts to implement short critical sections
  - Version 2.6 and later, fully preemptive
- Linux provides:
  - atomic integers
    - all math operations using atomic integers are performed without interruption
  - Semaphores
  - spinlocks
  - reader-writer versions of both semaphores and spinlocks
- On single-cpu system, spinlocks replaced by enabling and disabling kernel preemption

UNIX Concurrency Mechanisms

- UNIX provides a variety of mechanisms for interprocessor communication and synchronization including:



- Pipes, messages, and shared memory: used to communicate data between processes
- semaphores and signals: used to trigger actions by other processes

Solaris Synchronization

- Implements a variety of locks to support multitasking, multithreading (including real-time threads), and multiprocessing
- Uses **adaptive mutexes** for efficiency when protecting data from short code segments
  - Starts as a standard semaphore spin-lock
  - If lock held by a thread running on another CPU, spins
  - If lock held by non-run-state thread, block and sleep waiting for signal of lock being released
- Uses **condition variables**
- Uses **readers-writers locks** when longer sections of code need access to data
- Uses **turnstiles** to order the list of threads waiting to acquire either an adaptive mutex or reader-writer lock
  - Turnstiles are per-lock-holding-thread, not per-object
- Priority-inheritance per-turnstile gives the running thread the highest of the priorities of the threads in its turnstile

Pthreads Synchronization

- Pthreads API is OS-independent
- It provides:
  - mutex locks
  - condition variable
- Non-portable extensions include:
  - read-write locks
  - spinlocks